



American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science & Technology (FST)

Computer Graphics

Project Proposal

Flight Dash: Aircraft Control and Obstacle Avoidance

Submitted To

Dipta Justin Gomes

Submitted By

Group Number: k

Semester: FALL 2025-2026		Section: J
SN	Student Name	Student ID
01	NILOY PAUL	23-51773-2
02	ARNOB SARKER SUPTA	23-52080-2

1. Title:

Flight Dash: Aircraft Control and Obstacle Avoidance

2. Introduction:

Computer graphics plays a significant role in creating interactive digital environments, especially in fields such as simulation, visualization, and game development. Using graphical tools and programming libraries, developers can design engaging worlds that blend creativity with computational logic. Flight Dash: Aircraft Control and Obstacle Avoidance Simulation is a computer graphics-based game built using C++ and the OpenGL Utility Toolkit (GLUT). This project focuses on demonstrating essential graphics techniques through a sky-based aircraft game that integrates object animation, collision response.

The game places the player in control of a standard airplane flying over a city scenario. Although the main gameplay unfolds in the sky, the city structures including buildings, towers, and simple skyline elements serve as background to enhance visual depth and context. The aim of the game is straightforward yet engaging: the player must steer the airplane through the sky while avoiding bomb obstacles and collecting coins scattered along the flight path. These elements are placed dynamically to challenge the player's reaction time and decision-making skills.

From a technical standpoint, the project emphasizes the use of fundamental 2D graphics concepts. OpenGL primitives are used to design all visual components, including the airplane, bombs, coins, clouds, buildings, and background layers. The movement of the airplane is controlled using keyboard inputs, offering smooth navigation across the sky. Meanwhile, bombs descend or move depending on the game design logic, requiring the player to avoid them. Coins, on the other hand, function as a reward object that encourages exploration and improves player engagement. Keyboard events managing through GLUT allow for intuitive aircraft control, while some functions manage the movement of background elements, obstacles, and collectible items. Collision detection plays a central role in gameplay, if the airplane touches a bomb, the game will end. On the other hand, collecting a coin increases the score. These mechanics are implemented using geometric calculations based on object positions and boundaries.

The visual environment is designed to be simple yet appealing. The city backdrop provides a sense of depth, while scrolling clouds enhance the illusion of continuous forward movement. Although the project stays within the limits of 2D design, it effectively simulates motion and interaction through careful arrangement of shapes and animations.

Flight Dash: Aircraft Control and Obstacle Avoidance Simulation demonstrate how core concepts of computer graphics can be applied to create an interactive, enjoyable, and visually coherent game. By combining airplane control, bomb avoidance, and coin collection within a city-sky scenario, the project highlights essential skills in graphics programming, problem-solving, and visual design. It serves as both an educational exercise and a creative exploration of how OpenGL and GLUT can bring a simple 2D game world to life.

3. Description of the environment and list of methods:

The project is developed using C++, OpenGL, and GLUT, forming an efficient environment for building a 2D graphics-based game. The game window is configured with an orthographic projection to maintain a flat coordinate system suitable for drawing all elements. The environment includes a sky background, scrolling clouds, and a city skyline, all constructed using basic geometric primitives. Rendering in the project is performed using glBegin() and glEnd(), supported by OpenGL primitives such as:

- **GL_TRIANGLES:** used for shapes like airplane wings or bomb tips
- **GL_QUADS:** used for buildings, airplane body, and platforms
- **GL_LINES:** used for outlines or decorative elements
- **GL_POINTS:** for small effects or stars
- **GL_CIRCLE** (implemented using triangle fans): for clouds and coins

Also, there are variety of methods used to control the gameplay:

- **Display Function:** draws all objects on the screen using OpenGL primitives.
- **Keyboard Function:** Manages user input to move the airplane.
- **Update Function:** animate bombs, clouds, and background movement.
- **Collision detection Function:** checks when the airplane touches bombs or collects coins using distance and boundary formulas.
- **Drawing methods:**
 - **DrawAirplane()**
 - **DrawBomb()**
 - **DrawCoin()**
 - **DrawCloud()**
 - **DrawBuilding()**
- **glInit Function:** this function initializes the GLUT window, sets the display mode, window size.
- **myInit Function:** this function configures the initial OpenGL settings, including background color, projection mode, and coordinate system setup. It prepares the rendering environment before any objects are drawn.

4. Feature set:

Here there are 2 members (2x20=40). Minimum 40 features (Minimum 20 for each member):

Member 1 (NILOY PAUL): This member is responsible for designing, drawing, and animating the player-controlled airplane, bombs, and collectible coins using OpenGL primitives. Implements keyboard controls for smooth aircraft movement, collision detection with bombs and coins, real-time score tracking, and game-over/restart functionality. Manages all core gameplay logic to ensure responsive obstacle avoidance and rewarding coin collection mechanics.

These are some features:

1. Draw the main airplane using GL Primitives (like GL_TRIANGLES, GL_QUAD, etc.)
2. Color the airplane body
3. Airplane wings using GL_TRIANGLES
4. Cockpit window using a GL_QUAD
5. Smooth left movement of airplane on pressing 'A' or Left arrow
6. Smooth right movement on pressing 'D' or Right arrow
7. Boundary restriction: airplane cannot go above or below screen
8. Bombs using GL_TRIANGLES, GL_TRIANGLE_FAN (body)
9. Red flashing color for bombs
10. Random spawning of bombs from top of the screen
11. Downward movement animation of bombs
12. Coins as golden circles using triangle fan (GL_TRIANGLE_FAN)
13. Coin rotation animation
14. Random spawning of coins in the flight path
15. Coin collection increases score by 10
16. Play visual flash when coin is collected (simulated via color change)
17. Collision detection between airplane and bombs using bounding box
18. Game Over state when airplane collides with a bomb
19. Display "Game Over" text in the center when collision occurs
20. Score display in top-left corner (updates in real-time)
21. Restart game option after Game Over

Member 2 (ARNOB SARKER SUPTA): Responsible for building the entire visual environment, including gradient sky, parallax-scrolling clouds, detailed city skyline with varied buildings. Designs and animates decorative elements like the sun light, communication tower, runway, and dynamic smoke trail behind the airplane. Implements smooth continuous background scrolling to create the illusion of forward flight. Creates start screen, game-over screen, and real-time text display for score and instructions. Sets up OpenGL initialization (myInit) and manages the timer-based animation loop for 60 FPS performance. Ensures visual polish, depth, and immersive atmosphere throughout the game world.

These are some features:

1. Gradient sky background (light blue at top to darker blue at bottom)
2. Scrolling clouds using custom DrawCloud() with GL_TRIANGLE_FAN
3. Multiple cloud layers with different speeds for parallax scrolling effect
4. Random generation of 8–12 clouds at different heights and speeds
5. Continuous horizontal/vertical movement of clouds from right to left
6. Cloud respawns when they move out of the left/right side of the screen
7. City skyline at the bottom using multiple DrawBuilding()
8. Drawing at least 10 different buildings with varying heights and widths using GL_QUADS
9. Different colored windows on buildings (random yellow/white for lights)
10. Tall communication tower with antenna using GL_LINES and GL_QUADS
11. Scrolling city background (moves left to simulate forward flight)
12. Simple sun rays using thin GL_LINES
13. Stars (GL_POINTS) that appear when score > 100
14. Smoke trail behind the airplane using fading circles or quads
15. Smooth continuous forward motion illusion using layered scrolling
16. Ground at the very bottom using GL_QUADS
17. White dashed centerline on the road
18. Initial welcome screen with game title "Flight Dash" and "Press Space to Start"
19. myInit() – Set orthographic projection (glOrtho) and background color
20. myInit() – Enable smooth shading and point smoothing
21. Update function to control all animations at 60 FPS

5. Conclusion:

Flight Dash: Aircraft Control and Obstacle Avoidance Simulation successfully demonstrate the power of fundamental computer graphics concepts in creating an engaging, interactive 2D game using only C++, OpenGL, and GLUT. By combining basic geometric primitives, keyboard-driven animation, collision detection, and layered scrolling techniques, the project transforms simple shapes into a dynamic sky-based environment where the player controls an airplane, avoids descending bombs, and collects coins to increase their score. The game effectively illustrates core topics of the Computer Graphics course, including orthographic projection, real-time rendering, object transformation, parallax scrolling, and geometric collision detection while maintaining smooth performance and visual appeal within the constraints of 2D graphics.

The clear division of responsibilities between the two team members ensured balanced development, one focused on core gameplay mechanics and interactive objects, while the other concentrated on environmental design, background animation, and visual polish. Together, these efforts produced a cohesive and playable simulation that serves as both an educational tool and an entertaining game.

This game deliberately remains within the scope of basic OpenGL and 2D techniques, several enhancements could be implemented in future versions. These include adding sound effects, introducing power-ups, implementing multiple levels with increasing difficulty, incorporating particle systems for explosions, or extending the project to a side-scrolling 3D perspective using modern OpenGL. Additional obstacles such as birds or weather effects could also deepen gameplay. Despite its intentional simplicity, the project has no major technical limitations, performance remains stable even with multiple moving objects and parallax layers.